Robust Interpretation of Historical Documents in Knowledge Graphs Through Query Inference and Execution

Sebastià Nicolau[†], Adrià Molina [†] ✉, Oriol Ramos Terrades, and Josep Lladós

[1] Centre de Visió per Computador
[2] Universitat Autònoma de Barcelona
{snicolau, amolina, oriolrt, josep}@cvc.uab.cat

Abstract. The emergence of Large Language Models (LLMs) has redefined how users interact with information in digital environments. However, their widespread and often indiscriminate integration has raised significant concerns regarding reliability and trustworthiness issues that are particularly critical when accessing digital libraries and historical archives. How can one leverage the generalization capacity of an LLM without losing the level of accountability required for an archival institution? In this paper, we present an agentic retrieval system designed to deliver more accurate and verifiable access to historical data while preserving much of the flexibility associated with unconstrained LLMs. As a contribution to historical document analysis, we compare traditional Retrieval-Augmented Generation (RAG) with an agentic GraphRAG architecture in their ability to deliver historical information under realistic conditions, including the presence of OCR and transcription errors.
We introduce a semi-symbolic framework that integrates word-spotting techniques for post-OCR correction with a knowledge graph representation that enables the agent to access information through synthesized queries. The interleaved collaboration between word spotting and code generation allows the agent to construct strong retrieval queries that are robust to misinterpretation and hallucination, while still leveraging approximate search when noise and uncertainty, common in historical document analysis, would otherwise hinder precise retrieval.

Keywords: Historical Document Analysis · Document Analysis Systems · Graph-Based Document Analysis

1 Introduction

Historical Document Retrieval is the task that lies at the core of access portals to digital libraries and archives. Either by retrieving high-level features in content-based retrieval [27], or low-level features in writer identification [5] and word spotting [9], this task constitutes, for many users, the entry point to valuable historical sources and collections. These systems are central to the construction

[†]Both authors contributed equally to this work.

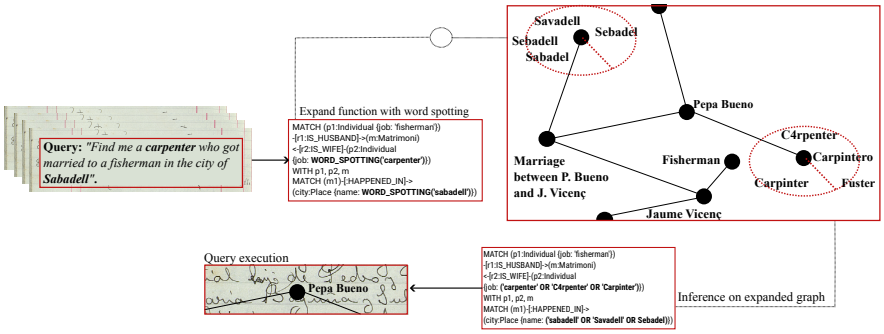


Fig. 1: Visual abstract of our contribution. A natural language request is translated into a Cypher[3] query. The model handles challenging node types (e.g., names, places) by choosing word-spotting expansion or exact matching. Queries run on the expanded graph, ensuring robust, flexible, and trustworthy retrieval.

of historical narratives [8,22], as they offer an inevitably partial view in which generalization and specificity must be balanced. [31].

While many industries have adopted neural-based representations integrated into their search engines [18], most GLAM institutions (Galleries, Libraries, Archives, and Museums) still rely on ontology and metadata-based approaches for organizing and accessing their information [10,11,21]. We attribute this apparent stagnation to the aforementioned tension: while analyzing historical data requires strong generalization capabilities due to its multi-domain, multi-source nature and wide temporal variance, many of the most interesting and important elements in historical documents are not discoverable through Pattern-Recognition methods. Rather, they are facts expressed through symbolic entities and relations, for which only an ontological approach can guarantee their presence or absence within a database. In recent years, agentic perspectives on language modeling have been shown to serve as a bridge between flexibility and robustness [28]; however, most of their applications are still found in digitally native documents [7]. The idea of leveraging a knowledge graph for an agent to index is appealing, but unrealistic in the presence of recognition artifacts and errors that can corrupt the inherent structure of such a graph. For an agentic (therefore generalizable), graph-based (therefore grounded) system to operate effectively in historical document analysis, it must account for inaccuracies in handwritten text recognition technologies, which may corrupt its structure and yield numerous false negatives.

In this work, we propose a seminal implementation of a language model agent that can access historical data by exploiting a historical knowledge graph via code execution. In contrast to common GraphRAG implementations [28], we construct the graph from historical handwritten documents; therefore, errors are not occasional corrupted pieces of information but a consistent characteristic

[3]Language designed for indexing graph databases.

of the graph. As shown in Figure 1, we integrate word-spotting techniques into the agentic process. In this way, we provide the code-generation model with the option to expand certain nodes of the graph to create a meta-node consisting of all possible variations of a given word. We validate this perspective using a combination of different HTR transcriptions, human transcriptions, multiple word-spotting techniques, diverse graph representations, and code generators on the Esposalles [26] dataset. Accordingly, we summarize our contributions as follows:

1. We introduce the first GraphRAG architecture designed explicitly for noisy historical knowledge graphs, where query execution is dynamically augmented through word-spotting–based literal expansion at runtime.
2. We contribute with the human annotation of 100 curated book-level questions on the Esposalles dataset, which serves as use case for our method, consisting of extractive tasks, statistical inference and multi-hop queries.

The rest of the work is organized as follows: in Section 2 an overview of Retrieval Augmented Generation literature is done, including its potential on managing digital archives. Our method is defined in Section 3, where we introduce from the query formulation to the code generation and word spotting injection. In Section 4 different data structures that will be employed are presented, and the Esposalles-DocVQA dataset is introduced as a showcase of our method's performance on a real environment. Results and conclusions are to be found in Sections 5 and 6 respectively.

2 Related Work

Pretrained generative models for automatic text generation based on Transformers rely solely on knowledge acquired during pretraining. This makes them prone to generating plausible but incorrect information, a phenomenon known as hallucination [17], and limits their ability to update knowledge dynamically. In contrast, traditional information retrieval systems efficiently provide relevant documents but lack the ability to synthesize or integrate this information into coherent and narrative outputs [12].

In 2020, Retrieval-Augmented Generation (RAG) systems were introduced [20]. These architectures combine an external retrieval process to obtain factual evidence with a generative model capable of incorporating this information into novel and coherent responses [20]. This approach aims to ensure factual fidelity in generated content while overcoming the limitations of purely generative or retrieval-based systems.

Early hybrid systems such as DrQA [4] combined retrieval with limited generation, often restricted to selecting textual fragments from retrieved documents. Later, REALM [14] represented a significant advancement by integrating dense retrieval during pretraining with generation, improving the semantic alignment between retrieved information and generated text. This approach was further extended by RAG, which explicitly leverages dense passage retrieval combined

with generative *Transformer*-based models such as BART [19], achieving a more coherent integration between retrieval and generation. Recent trends point to advanced hybrid approaches that combine traditional lexical and dense signals to enhance both the precision and coverage of RAG systems [15].

Embedding-based methods often struggle with multi-faceted textual queries [32]. Probabilistic approaches improve results [6] but still face challenges in reasoning over structured information, complex operations, and transparency in passage selection [3,16]. Agentic workflows address these issues by integrating reasoning and query actions. ReAct alternates step-by-step reasoning with queries to reduce factual errors [34], Self-Ask decomposes questions into subqueries and validates consistency [24], and Chain-of-Agents coordinates specialized LLMs for planning, retrieval, and verification in long contexts [35]. These approaches fall under Agentic RAG, which formalizes planning, reflection, and tool usage in the workflow [28]. Recent work at the intersection of vision and symbolic reasoning has attempted to bridge high-level semantic understanding with visual perception through program synthesis. Representative approaches include the neuro-symbolic framework VisProg and the code-based reasoning system ViperGPT. Both ViperGPT [30] and VisProg [13] provide zero-shot frameworks that answer visual queries by generating Python programs composed of API calls to pretrained perception models, such as object detectors and attribute classifiers. Instead of reasoning visually, these systems define the logic of visual checks and delegate execution, enabling interpretability, compositionality, and strong generalization without additional training.

In parallel, the integration of Knowledge Graphs (KGs) introduces a structured layer that facilitates multi-hop reasoning, allowing inference across multiple chains of relationships rather than only direct connections. GreaseLM demonstrates that combining a language model with a Graph Neural Network enhances entity inference [33]. KG^{2}RAG organizes and expands passages using an external KG [36], while GraphRAG generates graphs from free text and retrieves subgraphs as explainable evidence during generation [7]. Our work is situated within this context, employing a hybrid RAG approach that leverages routines to a graph database to produce reliable insights grounded in factual knowledge.

3 Method

Figure 2 overviews the proposed architecture that is inspired by the principles of Agentic GraphRAG architectures, the components of such architecture are:

Graph Construction (GraphDB) The graph construction process consists of injecting nodes and relations extracted from a handwritten historical corpus. This involves leveraging HTR and NER to identify people, places, events, and their interconnections within the texts. The extracted entities are modeled as nodes, while their semantic relationships are represented as edges. These structured elements are then ingested into a graph database, which serves as the backbone for indexing, enabling efficient querying, navigation, and knowledge discovery over the historical data.

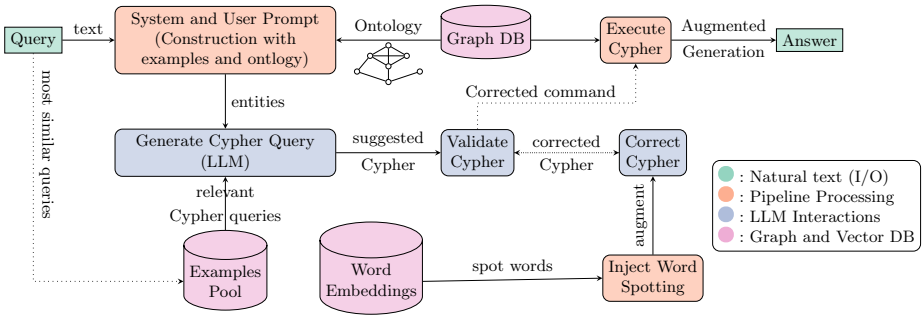


Fig. 2: Scheme of our proposed approach, including the word spotting injection step.

Ontology and Graph (System and User Prompt) Given any query, the system is provided with a schema that represents the ontology of the graph database being used. This schema, shall contain structured and ontological information about the database. The graph models the domain as a property graph in which nodes represent entities such as individuals, documents, events, and locations, each annotated with properties (e.g., name, year, file_name...) depending on the historical source. Relationships between nodes are explicitly typed (e.g., family relationships, locations...) and encode the semantics of how entities are connected within the dataset. The schema provided to the system is a human-readable, textual representation of this ontology. It describes node labels, relationship types, properties, and cardinalities, offering a high-level overview of the graph structure that enables the LLM to understand the underlying domain model, ensuring that generated relationship patterns and property usage are consistent with the actual topology of the graph.

Generate Cypher Query The system employs a zero-shot in-context learning paradigm to facilitate the translation from natural language questions to syntactically correct Cypher queries. A set of question-query pairs for each graph structure are elaborated, which serve as contextualized examples for the mapping between the user intent expressed in natural language and the corresponding Cypher logic. During query generation, these examples are injected into the prompt context alongside the graph schema and conversation history, allowing the LLM to identify analogous patterns between the input question and the provided examples. This approach enables the model to generate queries that adhere to established conventions for the specific database structure, including proper use of node labels, relationship directions, property filters, and result limiting clauses.

Validation Cypher The validation pipeline applies a multi-stage verification process with iterative error correction before query execution. Generated Cypher queries are first checked syntactically by submitting them to the graph database

planner without executing them, allowing the detection of malformed clauses or invalid constructs. They are then validated against the graph schema to ensure that relationship patterns conform to the predefined topology, automatically correcting relationship directions when necessary. The validation mechanism operates within a correction cycle controlled by an iteration counter. When validation fails, the system sends the query to an error correction phase where the LLM is prompted to fix the identified issues while preserving the query's semantic intent. The corrected query then re-enters validation for verification. This cycle continues until either validation succeeds without errors or a maximum iteration threshold is exceeded, preventing infinite correction loops.

Inject Word Spotting The Word Spotting module implements a query expansion mechanism that enriches generated Cypher queries by matching string literals against a glossary of textual variants obtained through word spotting. For each transcribed word, orthographic variants are clustered using an embedding-based similarity threshold, addressing a central challenge in historical document retrieval: OCR errors, spelling variability, and transcription inconsistencies that prevent exact string matching from retrieving relevant records. The expansion process is parameterized by an embedding specification that determines both the similarity metric (character-level or semantic) and the reference corpus (ground truth or OCR-derived text).

During query generation, the LLM marks literals that require approximate matching. At execution time, the system computes embeddings for each marked literal using either PHOC for character-level similarity or MPNet for semantic similarity, and retrieves similar variants from a vector database above a configurable threshold. The original Cypher query is then refactored by replacing marked literals with case-insensitive regular expressions covering all retrieved variants, while preserving the logical structure of the query. For complex inline patterns, matching conditions are extracted into dedicated `WHERE` clauses to ensure syntactic correctness. We formalize the word-spotting expansion operator as follows.

Formalization of the Expansion Operator. Let q denote the original Cypher query generated by the LLM. Let $L = \{l_1, \ldots, l_n\}$ be the set of string literals in q explicitly marked for expansion. For each literal $l \in L$, the word-spotting operator $\mathrm{WS}(\cdot)$ returns a set of textual variants:

$$\mathrm{WS}(l) \to \{v_1, \ldots, v_k\}.$$

We define the expanded query q' as the transformation of q in which every marked literal l is replaced by a regular expression matching the disjunction of its variants:

$$l \;\mapsto\; \texttt{regex}(v_1 \vee \cdots \vee v_k).$$

Thus, q' preserves the original logical structure of q while relaxing exact string constraints through embedding-based variant expansion.

Execute Cypher Query execution represents the final phase where the validated and potentially enriched Cypher statement is submitted to the graph database for actual data retrieval. The retrieved database records are fed alongside the original user question and the conversation history into a final LLM invocation that synthesizes a user-friendly response. The response generation is designed to present results concisely when documents are found, or provide direct, clear explanations when no relevant information exists, maintaining conversational context throughout multi-turn interactions.

4 Experimental Setup: A Use Case on the Esposalles Database

In this work, we make use of the Esposalles database [26] to validate the performance of our system in a real-world environment. Rather than using a standard Named Entity Recognition baseline, we will be employing a data set-up organized as follows.

4.1 The Esposalles Graph Database

Two approaches of graph modeling have been defined in this research. On one hand, in property-based graphs (see Figure 3, left) names and surnames are stored directly as properties of the individual nodes, resulting in a more compact graph structure with fewer nodes and relationships. On the other hand, Resource-Description Framework (see Figure 3, right) represents each individual by a unique identifier, while given names and surnames are modeled as distinct value nodes. Individuals are connected to these name nodes through explicit relationships, allowing names to be queried independently.

Esposalles Handwritten Recognition – We employ multiple Handwritten Text Recognition (HTR) models to generate noisy versions of the same database. This approach allows us to evaluate the robustness of our method against transcription errors and HTR inaccuracies. Specifically, we utilize off-the-shelf ViT-based HTR models [25] (0, 1, 2 and 4) trained to achieve different levels of accuracy on the Esposalles database, as summarized in Table 1a.

Word Embeddings – For word spotting, we generate word embeddings from both clean and noisy transcriptions. Table 1b illustrates how we leverage Pyramid Histogram of Characters (PHOC) embeddings [1] and a semantic search word model, MPNET [29], across HTR outputs with varying accuracies. This setup enables us to assess the robustness of different word spotting methods to transcription noise and to evaluate the relative advantages of each embedding approach in various scenarios.

Graph Databases – For each HTR transcription, we construct a corresponding graph database. In this work, we evaluate the document processing pipeline under the assumption of human-assisted graph construction, a common practice in many digital archives that already maintain interconnected sets of curated labels and entity annotations. Accordingly, we rely on ground-truth named

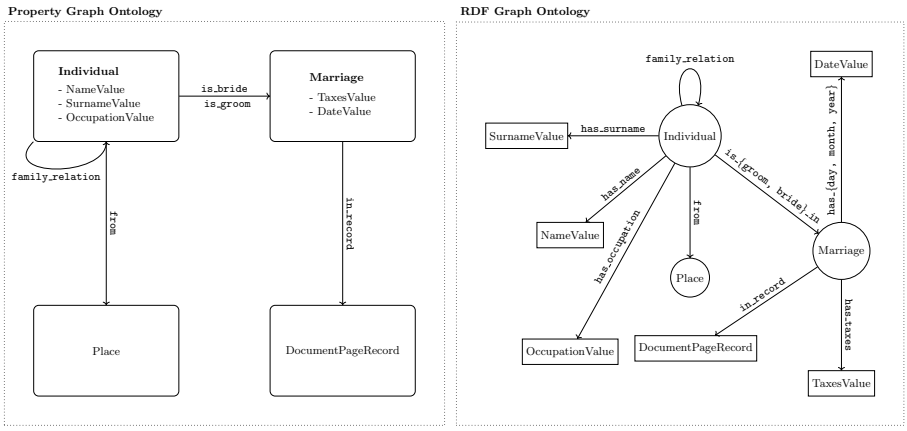


Fig. 3: Property graphs simplify Cypher queries, while RDF enables more complex searches via structured relationships.

entity annotations rather than performing automatic entity extraction. Our focus is query robustness under transcription noise, therefore automatic graph construction remains orthogonal and is left for future work. Table 1c details how we build property-based graphs using ground-truth transcriptions, representing perfect HTR accuracy. We also construct Resource Description Framework (RDF) graphs for HTR transcriptions at different accuracy levels, including ground truth, to assess whether the advantages of RDF graphs stem solely from perfect transcriptions or whether the approach remains robust under noisy conditions.

4.2 The Esposalles DocVQA Evaluation Dataset

A contribution of this work is the creation of an evaluation dataset comprising 100 manually curated question–answer pairs derived from the Esposalles records. The dataset is designed to evaluate the capabilities of the RAG systems proposed in this paper.

Questions were designed to reflect realistic analytical and lookup needs when working with structured archival records. Instead of focusing exclusively on direct fact retrieval, the dataset was intentionally designed to include questions that require different levels of aggregation, relational navigation, constraint filtering, and multi-step inference across records. This approach ensures coverage of both simple and operationally complex information needs that arise when interacting with historical registries.

All questions were strictly constrained to the information present in the Esposalles records and were validated to be answerable without external knowledge. Each reference answer was manually derived and verified against the source records to ensure correctness and reproducibility. This controlled construction process ensures that the evaluation measures how effectively each system retrieves relevant information and derives answers from the indexed material.

(a) HTR accuracy

Name	Acc. (%)
EsposallesGT	100.0
HTR4	98.0
HTR2	95.7
HTR1	78.3
HTR0	69.6

(b) Word embeddings

Embedding	Transcr.
MPNET	GT
MPNET	HTR4
MPNET	HTR2
MPNET	HTR1
MPNET	HTR0
PHOC	GT
PHOC	HTR4
PHOC	HTR2
PHOC	HTR1
PHOC	HTR0

(c) Graph databases

Name	Attr.
Property-GT	✗
RDF-GT	✓
RDF-HTR4	✓
RDF-HTR2	✓
RDF-HTR1	✓
RDF-HTR0	✓

Table 1: Overview of transcription accuracy, word embeddings, and graph configurations

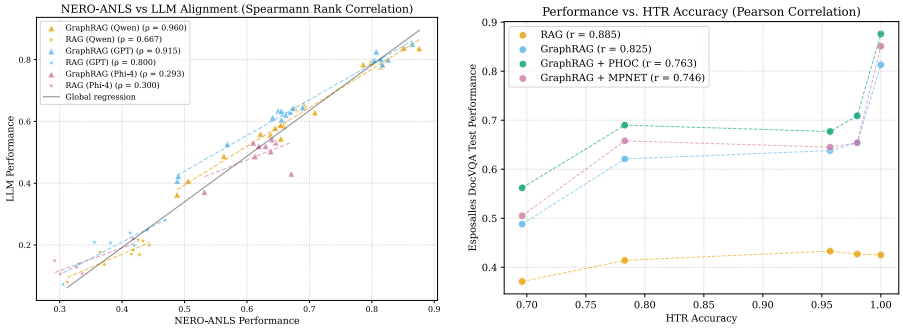
While traditional vector-based RAG systems perform well on semantic similarity tasks and direct entity lookups, they are not designed to answer reliably queries that require structured operations over multiple records or fields. The dataset was therefore designed to expose these differences in capability across retrieval and reasoning strategies. The dataset includes four main question types designed to test different retrieval and reasoning skills. Statistical aggregation questions (46%) involve counting, ranking, or analyzing distributions across records. Entity and relationship queries (31%) target specific entities, their attributes, and relationships, including temporal, spousal, and parental connections. Multi-hop reasoning questions (11%) require inference across multiple relationships, such as tracking inherited professions or generational patterns. Complex filtered queries (12%) combine multiple constraints, pattern matching, or range-based filters with aggregation, challenging the system to integrate multiple criteria for accurate retrieval.

5 Results

5.1 Metrics and evaluation

To assess the quality of generated answers across different RAG configurations, we employed a dual evaluation strategy combining deterministic similarity metrics with LLM-based semantic judgement. This approach balances the objectivity and reproducibility of automated metrics with the nuanced understanding of language models capable of handling linguistic variations inherent in historical documents.

Deterministic Entity-Based Evaluation The first evaluation metric is our proposed recall-oriented entity matching algorithm for historical document question-



(a) Alignment with LLM-as-a-Judge across HTR0–HTR4 and RDF/Property graphs.

(b) RAG, GraphRAG, and GraphRAG+Word Spotting performance under varying HTR accuracies.

Fig. 4: LLM-as-a-Judge alignment and system performance vs. HTR accuracy.

answering, termed NERO-ANLS (Named Entity – Recall-Oriented Average Normalized Levenshtein Similarity). It extends the original ANLS metric [23], which is unsuited for verbose generative models due to its sensitivity to extraneous text, by focusing on structured entity extraction and comparison. The deterministic procedure leverages the `ca_core_news_trf` named entity recognizer [2] to evaluate answer quality through entity-level similarity.

Entity Extraction Pipeline Evaluation begins by extracting key elements from both ground truth and generated answers, including numerical entities (dates, quantities, counts), named entities such as person names (PER), locations (LOC), organizations (ORG), and temporal expressions (DATE), as well as key lexical items like proper nouns, meaningful nouns (excluding stopwords), and profession terms. Text preprocessing includes markdown removal, normalization of whitespace, and case-folding. For expected answers (typically concise given the DocVQA dataset), we extract all meaningful words exceeding 2 characters to ensure proper coverage. For generated answers (often extensive and verbose), we focus on named entities and key nouns to identify relevant information.

Similarity Computation Similarity is computed in two stages. First, exact matching finds intersections between normalized expected and generated elements. Second, unmatched elements are compared using Levenshtein-distance-based fuzzy matching with a 0.6 threshold, accommodating common orthographic variations in historical transcriptions (e.g., “Pagès” vs. “pages”, “Terrassa” vs. “jerrassa”).

Score Calculation The final score prioritizes recall (coverage of expected information) over precision, reflecting the evaluation goal of assessing whether the

system retrieved all necessary information rather than penalizing verbosity:

$$\text{Score} = \begin{cases} 0.95 + 0.05 \times \text{recall} & \text{if recall} \geq 0.95 \\ 0.85 + 0.1 \times \text{recall} & \text{if } 0.95 > \text{recall} \geq 0.85 \\ F_\beta & \text{otherwise} \end{cases} \tag{1}$$

where F_β is the F-score with $\beta = 3$, where recall is weighted 3 times higher than precision, calculated as:

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}} \tag{2}$$

and precision is capped at a ratio of 1:5 (expected:generated elements) to avoid penalizing naturally verbose LLM responses. This design acknowledges that historical QA systems should prioritize information completeness over conciseness.

LLM-as-a-Judge Evaluation To complement the deterministic metric and capture semantic equivalences beyond lexical matching, we implemented an LLM-as-a-judge evaluation framework. This approach leverages an understanding of semantic similarity language from the model, context, and domain-specific knowledge to provide human-like quality assessments.

Evaluation Protocol and Scoring The LLM judge evaluates system answers against the ground truth using a structured prompt with the user question, reference, and generated answer. Assessment considers entity accuracy (names, places, numbers, dates, professions), completeness, correctness of counts, and minor orthographic variations. Scores range 0–10, with low for missing/incorrect entities, mid for partial correctness, and high for mostly correct answers; maximum scores require all key entities, minor spelling tolerance, and fully accurate counts, prioritizing coverage over style.

This dual evaluation framework is empirically validated by the strong correlation observed in Figure 4a. By combining deterministic quantitative scoring with qualitative LLM-based assessment, it provides a robust and reproducible evaluation of RAG system performance across diverse question types and varying levels of historical document complexity.

5.2 Quantitative Analysis

Once the experimental setup is defined, the global results are summarized in Table 2. First, we observe a strong alignment between the deterministic (NERO-ANLS) and the LLM-based evaluation protocols. Specifically, by computing the Spearman correlation coefficient, we obtain high correlation values (see Figure 4a), indicating that both evaluation protocols are well-suited for assessing the capabilities of RAG and GraphRAG systems on extractive and reasoning-based tasks, with some alignment issues when analyzing the lowest-performing models (Phi-4). Second, we note a clear performance ceiling for RAG systems,

			Qwen3-VL (32B)		GPT-5.1		Phi-4 (15B)	
			NERO-ANLS	**LLM**	**NERO-ANLS**	**LLM**	**NERO-ANLS**	**LLM**
RAG	GT Transcription		.435	.217	.469	.280	.418	.220
	HTR0 Transcription		.371	.137	.330	.139	.336	.108
	HTR1 Transcription		.414	.171	.412	.239	.292	.150
	HTR2 Transcription		.433	.213	.381	.207	.301	.107
	HTR4 Transcription		.427	.169	.419	.199	.327	.128
Graph RAG	-	Property-GT	.799	.791	.807	.826	.646	**.532**
		RDF-GT	.813	.778	**.865**	**.851**	.638	.503
		RDF-HTR1	.621	.560	.659	.626	.532	.372
	PHOC	Property-GT	.786	.784	.804	.798	.613	.531
		RDF-GT	**.876**	.836	.817	.784	.646	.542
		RDF-HTR1	.690	.653	.669	.631	.631	.520
	MPNET	Property-GT	.799	.817	.815	.801	.619	.520
		RDF-GT	.813	**.851**	.826	.800	.613	.487
		RDF-HTR1	.690	.658	.673	.643	**.671**	.431

Table 2: Performance metrics by system (rows) and back-end LLM (columns) for both the deterministic and the LLM-as-a-Judge metric. Bold font means **best performing method**.

which struggle to surpass the 0.50 threshold in terms of NERO-ANLS. This observation aligns closely with the characteristics of the Esposalles DocVQA Evaluation dataset introduced in Section 4.2, where ∼30% of the question–answer pairs correspond to entity and relationship queries. As will be further discussed in the qualitative analysis, this suggests that the performance of RAG systems is largely constrained to extractive, self-contained, and well-grounded tasks. Despite this limitation, RAG approaches exhibit significantly higher stability than the proposed GraphRAG method. This behavior can be interpreted either as a greater resilience of RAG systems to degradations in Handwritten Text Recognition quality or as an indication that GraphRAG systems are better positioned to exploit improvements in transcription accuracy. As shown in Figure 4b, although GraphRAG presents a steeper performance slope, the degradation of RAG performance remains relatively moderate as transcription noise increases.

Contrary to our initial hypothesis, word spotting does not stabilize performance by making HTR accuracy irrelevant. Instead, it consistently improves results across all systems, with a special impact on smaller models, with a large gap for Phi-4 word spotting (from 0.53 to 0.631 in NERO-ANLS when using noisy transcriptions from HTR1). PHOC-based representations are particularly effective for word spotting when compared to MPNet embeddings. Although MPNet is trained for semantic search, it still contributes to performance gains under lower HTR accuracy conditions. We attribute the overall improvement observed in PHOC-augmented GraphRAG systems to the intrinsic diversity and complexity of the Esposalles dataset. As is common in historical corpora, annotation inconsistencies arise due to high variability, contextual ambiguity, and subjective decisions made by annotators.

Within the same results table, we observe a consistent preference for our Word-Spotting–Augmented GraphRAG approach when using Resource Description Framework (RDF) graph modeling. Specifically, RDF achieves superior per-

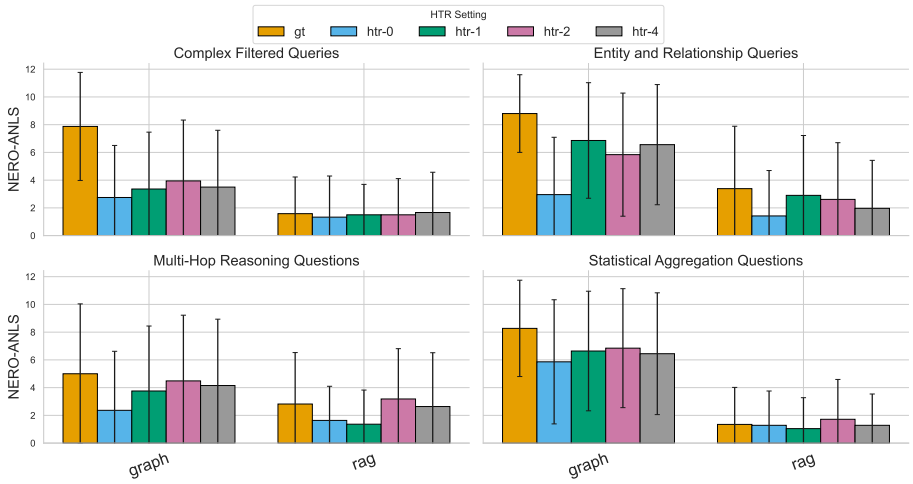


Fig. 5: Barplot for GraphRAG and RAG comparison on each question type and HTR transcription.

formance in 7 out of 9 NERO-ANLS comparisons (three LLMs indexed with three word-spotting alternatives in Table 2), with the only exceptions occurring when using Phi-4. Although property-based schemas offer a simpler structure that can facilitate the generation of syntactically correct queries, RDF-based representations provide greater expressiveness and are better suited to tasks that involve complex relational structures and long-range dependencies within the database. In this sense, the increased modeling complexity does not hinder performance; rather, it constitutes a valuable source of structured information that the model is able to exploit effectively.

With regard to the back-end LLM, which is responsible for both code generation and final answer production, we observe a substantial performance gap between `Phi-4` and `Qwen-3`. In contrast, the difference between `Qwen-3` and `GPT-5.1` is negligible. This suggests that code inference is, as expected, sensitive to model scale, particularly when moving from smaller to mid-sized models. However, performance appears to plateau beyond approximately 32B parameters, as no measurable improvement is observed on the evaluation dataset when further increasing model size. Additionally, in Figure 5, the results are disaggregated by question type and the HTR transcriptions used. As expected, Entity and Relationship Queries are the category in which RAG-based systems perform best. The largest performance gap, however, is observed in Statistical Aggregation Questions, which require the ability to analyze the collection holistically (a capability enabled by the graph-based nature of our method) and to perform mathematical operations, which is facilitated by the code-execution component of our approach. With respect to sensitivity to HTR accuracy, we observe a substantial performance gap between ground-truth and noisy transcriptions when

Table 3: Comparison of query results without and with PHOC expansion

Without PHOC Expansion	**With PHOC Expansion**
Question: How many individuals lived in Girona?	
Expected Answer: 9 individuals.	
Cypher Query: `MATCH (i:Individual) -[:LIVES]->(l:Place {name: "girona"}) RETURN COUNT(i)`	`MATCH (i:Individual) -[:LIVES]->(l:Place) WHERE l.name =~ '(?i)^(?:girona|gerona|gerona|gran|hieronma)$' RETURN COUNT(i)`
Database Records: `{'population': 5}`	`{'population': 9}`
Generated Answer: According to the results found, **5 individuals** lived in Girona.	Based on the information found, the number of individuals who lived in Girona is **9**.
Exact matching for "girona" misses variant spellings, while PHOC expansion generates a regex capturing all mis-transcriptions.	

addressing complex filtering questions. In contrast, multi-hop reasoning and statistical aggregation questions exhibit remarkable stability across varying transcription quality levels.

5.3 Qualitative Analysis

From a qualitative perspective, we present three illustrative examples demonstrating how our approach can improve the indexing and accessibility of digital libraries. First, in Table 3, we show how common agentic GraphRAG systems may distort historical records due to overreliance on their code inference step. In contrast, our method incorporates a word spotting stage, which in this example transforms the word "Girona" into a set of variations detected throughout the document, yielding a more accurate response. Second, in Table 4, we demonstrate that RAG-based systems still have potential to handle logical queries to some extent. However, a fundamental limitation of RAG is its design constraint: it typically contains only a subset of the documents, which restricts holistic analysis. Achieving full-context reasoning with RAG would require a large context window and significant computational resources. In contrast, graph-based systems, whether combined with code execution or not, enable holistic analysis of document collections more efficiently and comprehensively.

Table 4: Comparison of query results between GraphRAG and RAG approaches

RAG Approach	**GraphRAG Approach**
Question: Where do individuals with the surname Colomer live?	
Expected Answer: Barcelona, Arenys, Pierola, and Viladecans.	
Query/Method: Retrieved N document fragments mentioning Colomer (limited sample)	`MATCH (p:Individual) -[:SURNAME1]->(surname:value) MATCH (p)-[:LIVES]->(l:Place) WHERE surname.name =~ '(?i)^(?:colomer|...)$' RETURN l.name, COUNT(p)`
Generated Answer: Individuals with the surname Colomer, live in **Bara**. Specifically, Hieronym Colomer, a tailor from Ribes, lives in Bara.	Mainly in **Barcelona**, where there are the most records (5), **Arenys**, **Pierola** and **Viladecans**.
RAG retrieves only a limited set of documents, producing partial answers, while GraphRAG queries all nodes for full coverage.	

6 Conclusions

We presented a framework integrating agentic paradigms and word spotting into Historical Document Analysis, bridging the gap between digitally native and historical documents in graph-based systems. Experiments on the Esposalles DocVQA dataset demonstrate that word spotting improves robustness to noisy transcriptions, particularly when combined with RDF graphs, while system performance stabilizes once sufficient model capabilities are reached.

Our evaluation shows strong alignment between quantitative metrics and LLM-based assessment, validating both accuracy and interpretability. Even with ground-truth transcriptions, PHOC-based word spotting remains beneficial due to spelling variability and inconsistent annotations. By consolidating orthographic variants within graph nodes, the approach ensures traceable, auditable retrieval, providing holistic and trustworthy access to historical information. These results suggest that agentic graph-based workflows are a promising direction for reliable historical document analysis.

Acknowledgements

This work has been partially supported by the Spanish project PID2024-157778OB-I00, Ministerio de Ciencia e Innovación, the Departament de Cultura of the Generalitat de Catalunya, and the CERCA Program / Generalitat de Catalunya. Adrià Molina is funded with the PRE2022-101575 grant provided by MCIN / AEI / 10.13039 / 501100011033 and by the European Social Fund (FSE+).

References

1. Almazán, J., Gordo, A., Fornés, A., Valveny, E.: Word spotting and recognition with embedded attributes. IEEE transactions on pattern analysis and machine intelligence (2014)
2. Armengol-Estapé, J., Carrino, C.P., Rodriguez-Penagos, C., de Gibert Bonet, O., Armentano-Oller, C., Gonzalez-Agirre, A., Melero, M., Villegas, M.: Are multilingual models the best choice for moderately under-resourced languages? In: ACL-IJCNLP 2021. Association for Computational Linguistics (2021)
3. Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., Van Den Driessche, G.B., Lespiau, J.B., Damoc, B., Clark, A., et al.: Improving language models by retrieving from trillions of tokens. In: ICML. pp. 2206–2240. PMLR (2022)
4. Chen, D., Fisch, A., Weston, J., Bordes, A.: Reading wikipedia to answer open-domain questions (2017)
5. Christlein, V., Nicolaou, A., Seuret, M., Stutzmann, D., Maier, A.: Icdar 2019 competition on image retrieval for historical handwritten documents
6. Chun, S.: Improved probabilistic image-text representations (2023)
7. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph rag approach to query-focused summarization (2024)
8. Epstein, R., Li, J.: Can biased search results change people's opinions about anything at all? a close replication of the search engine manipulation effect (seme). Plos one **19**(3), e0300727 (2024)
9. Fischer, A., Frinken, V., Fornés, A., Bunke, H.: Transcription alignment of latin manuscripts using hidden markov models. In: Proceedings of the 2011 Workshop on Historical Document Imaging and Processing. pp. 29–36 (2011)
10. French Government / Archives de France: France archives – advanced search (2026), `https://francearchives.gouv.fr/en/advancedSearch`, accessed: 2026-02-10
11. Generalitat de Catalunya, Departament de Cultura: Arxius en línia – cercador avançat (2026), `https://arxiusenlinia.cultura.gencat.cat/#/cercaavancada/cerca`, accessed: 2026-02-10
12. Gienapp, L., Scells, H., Deckers, N., Bevendorff, J., Wang, S., Kiesel, J., Syed, S., Fröbe, M., Zuccon, G., Stein, B., et al.: Evaluating generative ad hoc information retrieval. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval
13. Gupta, T., Kembhavi, A.: Visual programming: Compositional visual reasoning without training. In: Proceedings of the IEEE/CVF CVPR
14. Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.: Retrieval augmented language model pre-training. In: ICML. pp. 3929–3938. PMLR (2020)
15. Hsu, H.L., Tzeng, J.: Dat: Dynamic alpha tuning for hybrid retrieval in retrieval-augmented generation (2025)
16. Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., Grave, E.: Atlas: Few-shot learning with retrieval augmented language models. Journal of Machine Learning Research **24**(251), 1–43 (2023)
17. Kalai, A.T., Nachum, O., Vempala, S.S., Zhang, E.: Why language models hallucinate (2025)

18. Karakurt, E., Akbulut, A.: Retrieval augmented generation (rag) and large language models (llms) for enterprise knowledge management and document automation: A systematic literature review (2025)
19. Lewis, M.e.: Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: Proceedings of the 58th annual meeting of the association for computational linguistics (2020)
20. Lewis, P.e.a.: Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)
21. Library of Congress: Library of congress catalog search (2026), `https://search.catalog.loc.gov/search`, accessed: 2026-02-10
22. Ludolph, R., Allam, A., Schulz, P.J.: Manipulating google's knowledge graph box to counter biased information processing during an online search on vaccination. Journal of medical Internet research (2016)
23. Mathew, M., Karatzas, D., Jawahar, C.: Docvqa: A dataset for vqa on document images. In: Proceedings of the IEEE/CVF winter conference on applications of computer vision. pp. 2200–2209 (2021)
24. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models (2022)
25. Rodríguez, A.M., Terrades, O.R., Lladós, J.: The ocr quest for generalization: Learning to recognize low-resource alphabets with model editing (2025)
26. Romero, V., Fornés, A., Serrano, N., Sánchez, J.A., Toselli, A.H., Frinken, V., Vidal, E., Lladós, J.: The esposalles database: An ancient marriage license corpus for off-line handwriting recognition. Pattern Recognition **46**(6), 1658–1669 (2013)
27. Seuret, M., Nicolaou, A., Rodríguez-Salas, D., Weichselbaumer, N., Stutzmann, D., Mayr, M., Maier, A., Christlein, V.: Icdar 2021 competition on historical document classification. In: International Conference on Document Analysis and Recognition
28. Singh, A., Ehtesham, A., Kumar, S., Khoei, T.T.: Agentic retrieval-augmented generation: A survey on agentic rag (2025)
29. Song, K., Tan, X., Qin, T., Lu, J., Liu, T.Y.: Masked and permuted pre-training for language understanding. Advances in neural information processing systems (2020)
30. Surís, D., Menon, S., Vondrick, C.: Vipergpt: Visual inference via python execution for reasoning. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 11888–11898 (2023)
31. Wang, C., Liu, X., Yue, Y., Tang, X., Zhang, T., Jiayang, C., Yao, Y., Gao, W., Hu, X., Qi, Z., et al.: Survey on factuality in large language models: Knowledge, retrieval and domain-specificity (2023)
32. Wang, H., Zhan, Y., Liu, L., Ding, L., Yu, J.: Balanced similarity with auxiliary prompts: towards alleviating text-to-image retrieval bias for clip in zero-shot learning
33. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A comprehensive survey on graph neural networks. IEEE transactions on neural networks and learning systems **32**(1), 4–24 (2020)
34. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. In: ICLR (2023)
35. Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., Arik, S.: Chain of agents: Large language models collaborating on long-context tasks. Advances in Neural Information Processing Systems (2024)
36. Zhu, X., Xie, Y., Liu, Y., Li, Y., Hu, W.: Knowledge graph-guided retrieval augmented generation (2025)